

DESIGN AND ANALYSIS OF ALGORITHMS

MICRO PROJECT

LIFT OPTIMIZATION

CODE:

```
import java.util.*;

class ScanAlgorithm {

    public static List<Integer> optimizePath(int currentFloor, List<Integer>
requests, String direction) {

        List<Integer> upList = new ArrayList<>();
        List<Integer> downList = new ArrayList<>();

        // تقسيم requests into up and down
        for (int request : requests) {
            if (request > currentFloor) {
                upList.add(request);
            } else if (request < currentFloor) {
                downList.add(request);
            }
        }

        Collections.sort(upList); // ascending
        Collections.sort(downList, Collections.reverseOrder()); // descending

        List<Integer> result = new ArrayList<>();
```

```

// LOOK logic
if (direction.equalsIgnoreCase("UP")) {
    result.addAll(upList);
    result.addAll(downList);
} else {
    result.addAll(downList);
    result.addAll(upList);
}

return result;
}
}

// Simulation Class
class ElevatorSimulation {

    private List<Integer> optimizedPath;
    private int currentSimFloor;

    public ElevatorSimulation(int startFloor, List<Integer> requests, String
direction) {
        this.currentSimFloor = startFloor;
        this.optimizedPath = ScanAlgorithm.optimizePath(startFloor,
requests, direction);
    }
}

```

```

public void startSimulation() {
    new Thread(() -> {
        try {
            for (int nextFloor : optimizedPath) {

                String currentDir = (nextFloor > currentSimFloor) ? "UP" :
"DOWN";

                // Move step by step
                while (currentSimFloor != nextFloor) {
                    Thread.sleep(800);

                    if (currentSimFloor < nextFloor) {
                        currentSimFloor++;
                    } else {
                        currentSimFloor--;
                    }

                    System.out.println("Moving " + currentDir + " | Floor: " +
currentSimFloor);
                }

                // Stop at floor
                System.out.println("STOPPED at Floor " + currentSimFloor +
" | DOORS OPEN");
                Thread.sleep(1500);
            }
        }
    }
}

```

```
        System.out.println("DOORS CLOSING...");
        Thread.sleep(500);
    }

    System.out.println("All requests processed!");

    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}).start();
}
}
```

// Main Class (Only public class)

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter current floor: ");
```

```
        int currentFloor = sc.nextInt();
```

```
        System.out.print("Enter number of requests: ");
```

```
        int n = sc.nextInt();
```

```
List<Integer> requests = new ArrayList<>();  
System.out.println("Enter requested floors:");  
for (int i = 0; i < n; i++) {  
    requests.add(sc.nextInt());  
}  
  
System.out.print("Enter direction (UP/DOWN): ");  
String direction = sc.next();  
  
ElevatorSimulation sim = new ElevatorSimulation(currentFloor,  
requests, direction);  
sim.startSimulation();  
  
sc.close();  
}  
}
```

Output

Clear

```
Enter current floor: 5
Enter number of requests: 3
Enter requested floors:
3 4 5
Enter direction (UP/DOWN): down
Moving DOWN | Floor: 4
STOPPED at Floor 4 | DOORS OPEN
DOORS CLOSING...
Moving DOWN | Floor: 3
STOPPED at Floor 3 | DOORS OPEN
DOORS CLOSING...
All requests processed!

=== Code Execution Successful ===
```

